Matplotlib Full Course — Beginner to Advanced

Python | Bangla Explanation | Practical Examples

1. Introduction

Matplotlib হলো Python-এর data visualization library। এটি দিয়ে numerical data থেকে charts, graphs এবং scientific visualizations তৈরি করা যায়।

Uses

- Data Analysis
- Exploratory Data Analysis (EDA)
- Scientific Computing
- Business Reports
- Machine Learning Visualization
- Statistical Charts

Installation

```
pip install matplotlib numpy pandas
```

Import

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

2. Figure and Axes

Matplotlib-এর দুটি প্রধান concept:

- Figure: পুরো drawing canvas।
- Axes: যেখানে actual graph তৈরি হয়।

```
fig, ax = plt.subplots()

ax.plot([1, 2, 3], [10, 20, 30])

ax.set_title("My First Graph")

plt.show()
```

3. Basic Line Plot

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 30, 25]

plt.plot(x, y)

plt.title("Line Chart")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")

plt.show()
```

Customize Line

```
plt.plot(
    x,
    y,
    color="blue",
    linewidth=2,
    linestyle="--",
    marker="o",
    markersize=8,
    label="Sales"
)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Revenue")

plt.legend()
plt.grid(True)

plt.show()
```

Line Styles

Style	Meaning
`-`	Solid
`--`	Dashed
`:`	Dotted
`-.`	Dash-dot

Markers

Marker	Meaning
`o`	Circle
`s`	Square
`^`	Triangle
`*`	Star
`x`	Cross
`+`	Plus

4. Multiple Line Charts

```
months = ["Jan", "Feb", "Mar", "Apr"]

product_a = [100, 150, 120, 200]
product_b = [80, 130, 170, 190]

plt.plot(months, product_a, label="Product A")
plt.plot(months, product_b, label="Product B")

plt.title("Product Comparison")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.legend()
plt.show()
```

5. Bar Chart

Categorical data compare করার জন্য ব্যবহার হয়।

```
languages = ["Python", "Java", "C++", "JavaScript"]
students = [100, 80, 60, 90]

plt.bar(languages, students)

plt.title("Language Popularity")
plt.xlabel("Language")
plt.ylabel("Students")

plt.show()
```

Customize Bar Chart

```
bars = plt.bar(
    languages,
    students,
    width=0.5,
    edgecolor="black"
)

plt.bar_label(bars)

plt.show()
```

6. Horizontal Bar Chart

```
plt.barh(languages, students)

plt.title("Horizontal Bar Chart")
plt.xlabel("Students")

plt.show()
```

7. Grouped Bar Chart

```
x = np.arange(3)

year_2025 = [80, 60, 50]
year_2026 = [100, 70, 65]

width = 0.35

fig, ax = plt.subplots()

ax.bar(x - width / 2, year_2025, width, label="2025")
ax.bar(x + width / 2, year_2026, width, label="2026")

ax.set_xticks(x)
ax.set_xticklabels(["Python", "Java", "C++"])

ax.legend()

plt.show()
```

8. Stacked Bar Chart

```
categories = ["Jan", "Feb", "Mar"]

product_a = [100, 150, 120]
product_b = [50, 70, 80]

plt.bar(categories, product_a, label="Product A")

plt.bar(
    categories,
    product_b,
    bottom=product_a,
    label="Product B"
)

plt.legend()
plt.show()
```

9. Scatter Plot

দুটি numerical variables-এর relationship দেখায়।

```
height = [150, 160, 170, 180, 190]
weight = [50, 55, 65, 75, 85]

plt.scatter(height, weight)

plt.title("Height vs Weight")
plt.xlabel("Height")
plt.ylabel("Weight")

plt.show()
```

Customize Scatter

```
plt.scatter(
    height,
    weight,
    s=100,
    color="red",
    alpha=0.7
)

plt.show()
```

Color Mapping

```
rng = np.random.default_rng(42)

x = rng.random(50)
y = rng.random(50)
colors = np.arange(50)

plt.scatter(
    x,
    y,
    c=colors,
    cmap="viridis"
)

plt.colorbar()
plt.show()
```

10. Histogram

Numerical data-এর frequency distribution দেখায়।

```
marks = [50, 60, 70, 80, 60, 70, 90, 80, 75, 65]

plt.hist(
    marks,
    bins=5,
    edgecolor="black"
)

plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Frequency")

plt.show()
```

Density Histogram

```
plt.hist(marks, bins=5, density=True)

plt.show()
```

Difference:

- Bar chart: Categorical data।
- Histogram: Numerical distribution।

11. Pie Chart

```
labels = ["Python", "Java", "C++", "JavaScript"]
values = [40, 25, 15, 20]

plt.pie(
    values,
    labels=labels,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Language Distribution")
plt.axis("equal")

plt.show()
```

Explode

```
explode = [0.1, 0, 0, 0]

plt.pie(
    values,
    labels=labels,
    explode=explode,
    autopct="%1.1f%%"
)

plt.show()
```

12. Box Plot

Distribution, median এবং potential outliers দেখায়।

```
data = [10, 20, 25, 30, 35, 40, 45, 100]

plt.boxplot(data)

plt.title("Box Plot")

plt.show()
```

Multiple Box Plots

```
data = [
    [10, 20, 30, 40],
    [20, 30, 40, 50],
    [30, 40, 50, 60]
]

plt.boxplot(
    data,
    tick_labels=["A", "B", "C"]
)
```

```
plt.show()
```

Box plot components:

- Median
- Q1
- Q3
- IQR
- Whiskers
- Fliers (potential outliers)

IQR = Q3 - Q1

Common outlier boundaries:

```
Lower = Q1 - 1.5 * IQR
Upper = Q3 + 1.5 * IQR
```

13. Violin Plot

Distribution-এর density shape দেখায়।

```
data = [
    [10, 20, 25, 30, 35, 40],
    [20, 25, 30, 35, 40, 45]
]

plt.violinplot(data)

plt.title("Violin Plot")

plt.show()
```

14. Area Chart

```
months = ["Jan", "Feb", "Mar", "Apr"]
sales = [100, 150, 120, 200]

plt.fill_between(
    months,
    sales,
    alpha=0.4
)

plt.plot(months, sales)

plt.title("Sales Area Chart")

plt.show()
```

Stacked Area

```
x = [1, 2, 3, 4]

a = [10, 20, 15, 25]
b = [15, 10, 20, 15]
```

```
plt.stackplot(
    x,
    a,
    b,
    labels=["Product A", "Product B"]
)

plt.legend()
plt.show()
```

15. Step and Stem Plot

Step Plot

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 30, 25]

plt.step(x, y, where="mid")

plt.show()
```

Stem Plot

```
x = np.arange(1, 10)
y = x ** 2

plt.stem(x, y)

plt.show()
```

16. Error Bar

Uncertainty বা error range দেখায়।

```
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]
errors = [2, 3, 1, 4]

plt.errorbar(
    x,
    y,
    yerr=errors,
    fmt="o",
    capsize=5
)

plt.show()
```

17. Heatmap

```
data = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
```

```
])

fig, ax = plt.subplots()

image = ax.imshow(data, cmap="viridis")

fig.colorbar(image)

ax.set_title("Heatmap")

plt.show()
```

Add Values

```
fig, ax = plt.subplots()

image = ax.imshow(data, cmap="viridis")

for i in range(data.shape[0]):
    for j in range(data.shape[1]):
        ax.text(
            j,
            i,
            data[i, j],
            ha="center",
            va="center"
        )

fig.colorbar(image)

plt.show()
```

18. Subplots

একটি Figure-এর মধ্যে একাধিক chart তৈরি করা যায়।

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

axes[0, 0].plot([1, 2, 3], [10, 20, 30])
axes[0, 0].set_title("Line Plot")

axes[0, 1].bar(["A", "B", "C"], [10, 20, 30])
axes[0, 1].set_title("Bar Chart")

axes[1, 0].scatter([1, 2, 3], [30, 20, 10])
axes[1, 0].set_title("Scatter Plot")

axes[1, 1].hist([10, 20, 20, 30, 30, 30])
axes[1, 1].set_title("Histogram")

plt.tight_layout()
plt.show()
```

Different Layouts

```
fig, axes = plt.subplots(1, 3)
```

```
fig, axes = plt.subplots(3, 1)
```

19. Figure Size and DPI

```
fig, ax = plt.subplots(figsize=(10, 5), dpi=150)

ax.plot([1, 2, 3], [10, 20, 30])

plt.show()
```

- `figsize` : Figure size in inches।
- `dpi` : Dots per inch; output resolution নিয়ন্ত্রণ করে।

20. Axis Customization

X and Y Limits

```
plt.plot([1, 2, 3, 4], [10, 20, 30, 40])

plt.xlim(0, 5)
plt.ylim(0, 50)

plt.show()
```

X and Y Ticks

```
plt.xticks([1, 2, 3, 4])
plt.yticks([0, 10, 20, 30, 40])
```

Rotate Labels

```
plt.xticks(rotation=45)
```

Custom Tick Labels

```
fig, ax = plt.subplots()

ax.plot([1, 2, 3], [10, 20, 30])

ax.set_xticks([1, 2, 3])
ax.set_xticklabels(["Jan", "Feb", "Mar"])

plt.show()
```

21. Grid and Legend

Grid

```
plt.plot([1, 2, 3], [10, 20, 30])

plt.grid(
    True,
    linestyle="--",
    alpha=0.5
```

```
)

plt.show()
```

Legend

```
plt.plot([1, 2, 3], [10, 20, 30], label="Sales")

plt.legend(
    loc="upper left",
    title="Category"
)

plt.show()
```

22. Annotations and Text

Graph-এর নির্দিষ্ট point-এ text যোগ করা যায়।

```
x = [1, 2, 3, 4]
y = [10, 20, 15, 30]

fig, ax = plt.subplots()

ax.plot(x, y, marker="o")

ax.annotate(
    "Highest",
    xy=(4, 30),
    xytext=(3, 35),
    arrowprops={"arrowstyle": "->"}
)

plt.show()
```

Add Text

```
plt.text(
    2,
    20,
    "Important Point"
)
```

23. Horizontal and Vertical Lines

Horizontal Line

```
plt.plot([1, 2, 3], [10, 20, 30])

plt.axhline(
    y=20,
    linestyle="--",
    label="Average"
)
```

```
plt.legend()
plt.show()
```

Vertical Line

```
plt.axvline(x=2, linestyle=":")
```

Reference Line

```
plt.axhline(y=50)
plt.axvline(x=3)
```

24. Logarithmic Scale

যখন data values-এর range অনেক বড় হয় তখন logarithmic scale useful।

```
x = np.arange(1, 6)
y = [10, 100, 1000, 10000, 100000]

fig, ax = plt.subplots()

ax.plot(x, y)

ax.set_yscale("log")

plt.show()
```

Other scales:

```
ax.set_xscale("log")
ax.set_yscale("linear")
```

25. Multiple Y-Axes

দুটি ভিন্ন scale-এর data একই graph-এ দেখাতে `twinx()` ব্যবহার হয়।

```
x = [1, 2, 3, 4]

temperature = [25, 28, 30, 27]
sales = [100, 200, 300, 250]

fig, ax1 = plt.subplots()

ax1.plot(x, temperature)
ax1.set_ylabel("Temperature")

ax2 = ax1.twinx()

ax2.plot(x, sales, linestyle="--")
ax2.set_ylabel("Sales")

plt.show()
```

26. Colormaps

Colormap values অনুযায়ী colors assign করে।

```
x = np.random.default_rng(42).random(50)
y = np.random.default_rng(43).random(50)
values = np.arange(50)

plt.scatter(
    x,
    y,
    c=values,
    cmap="plasma"
)

plt.colorbar()

plt.show()
```

Common colormaps:

- viridis
- plasma
- inferno
- magma
- coolwarm
- cividis
- gray

27. Mathematical Function Plotting

```
x = np.linspace(-10, 10, 200)

y = x ** 2

plt.plot(x, y)

plt.title("Y = X²")

plt.show()
```

Sine Wave

```
x = np.linspace(0, 2 * np.pi, 200)

y = np.sin(x)

plt.plot(x, y)

plt.title("Sine Wave")

plt.show()
```

Multiple Functions

```
x = np.linspace(-5, 5, 200)

plt.plot(x, x ** 2, label="x²")
```

```
plt.plot(x, x ** 3, label="x³")
plt.plot(x, np.abs(x), label="|x|")

plt.legend()
plt.show()
```

28. Saving Figures

Save PNG

```
plt.plot([1, 2, 3], [10, 20, 30])

plt.savefig("chart.png")

plt.show()
```

Save High Resolution

```
plt.savefig(
    "chart.png",
    dpi=300,
    bbox_inches="tight"
)
```

Save PDF

```
plt.savefig("chart.pdf")
```

Transparent Background

```
plt.savefig(
    "chart.png",
    transparent=True
)
```

Best practice: `savefig()` `show()`-এর আগে call করো।

29. Matplotlib with NumPy

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(1, 11)
y = x ** 2

plt.plot(x, y)

plt.title("NumPy Data Visualization")

plt.show()
```

30. Matplotlib with Pandas

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.DataFrame({
    "month": ["Jan", "Feb", "Mar", "Apr"],
    "sales": [100, 150, 120, 200]
})

plt.plot(df["month"], df["sales"])

plt.title("Monthly Sales")

plt.show()
```

Pandas Built-in Plotting

```
df.plot(
    x="month",
    y="sales",
    kind="bar"
)

plt.show()
```

Pandas-এর plotting backend হিসেবে Matplotlib ব্যবহার করা হয় by default।

31. Working with Dates

```
import pandas as pd
import matplotlib.pyplot as plt

dates = pd.date_range(
    start="2026-01-01",
    periods=5,
    freq="D"
)

sales = [100, 150, 120, 200, 250]

fig, ax = plt.subplots()

ax.plot(dates, sales, marker="o")

fig.autofmt_xdate()

plt.show()
```

32. Styles

Matplotlib-এর built-in styles ব্যবহার করা যায়।

```
print(plt.style.available)
```

Example:

```
plt.style.use("ggplot")

plt.plot([1, 2, 3], [10, 20, 30])
```

```
plt.show()
```

Style temporarily ব্যবহার:

```
with plt.style.context("ggplot"):
    plt.plot([1, 2, 3], [10, 20, 30])
    plt.show()
```

33. Interactive Plotting

Interactive display-এর জন্য Jupyter Notebook-এ ব্যবহার করা যায়:

```
%matplotlib widget
```

এটির জন্য প্রয়োজন হতে পারে:

```
pip install ipympl
```

Notebook-এর বাইরে interactive applications তৈরি করতে Matplotlib-এর event handling এবং animation API-ও শেখা যায়।

34. Animation

Matplotlib দিয়ে animated charts তৈরি করা যায়।

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

fig, ax = plt.subplots()

x = np.linspace(0, 2 * np.pi, 100)

line, = ax.plot(x, np.sin(x))

ax.set_ylim(-1.5, 1.5)

def update(frame):
    line.set_ydata(np.sin(x + frame * 0.1))
    return line,

animation = FuncAnimation(
    fig,
    update,
    frames=100,
    interval=50,
    blit=True
)

plt.show()
```

Animation save করতে FFmpeg বা Pillow-এর মতো writer প্রয়োজন হতে পারে।

35. 3D Plotting

3D chart তৈরি করতে `projection="3d"` ব্যবহার করা হয়।

```
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure()

ax = fig.add_subplot(
    111,
    projection="3d"
)

x = [1, 2, 3, 4]
y = [2, 3, 4, 5]
z = [10, 20, 30, 40]

ax.plot(x, y, z)

ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")

plt.show()
```

3D Scatter

```
fig = plt.figure()

ax = fig.add_subplot(111, projection="3d")

ax.scatter(x, y, z)

plt.show()
```

3D Surface

```
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)

X, Y = np.meshgrid(x, y)

Z = np.sin(np.sqrt(X ** 2 + Y ** 2))

fig = plt.figure()

ax = fig.add_subplot(111, projection="3d")

ax.plot_surface(X, Y, Z, cmap="viridis")

plt.show()
```

36. Statistical Visualization

Normal Distribution

```
rng = np.random.default_rng(42)
```

```
data = rng.normal(
    loc=50,
    scale=10,
    size=1000
)

plt.hist(
    data,
    bins=30,
    density=True
)

plt.title("Normal Distribution")

plt.show()
```

Cumulative Histogram

```
plt.hist(
    data,
    bins=30,
    cumulative=True
)

plt.show()
```

Correlation Visualization

```
data = np.array([
    [1, 0.8, 0.3],
    [0.8, 1, 0.5],
    [0.3, 0.5, 1]
])

fig, ax = plt.subplots()

image = ax.imshow(
    data,
    cmap="coolwarm",
    vmin=-1,
    vmax=1
)

fig.colorbar(image)

plt.show()
```

37. Object-Oriented vs Pyplot Interface

Pyplot Interface

```
plt.plot([1, 2, 3], [10, 20, 30])

plt.title("Graph")
```

```
plt.show()
```

Object-Oriented Interface

```
fig, ax = plt.subplots()

ax.plot([1, 2, 3], [10, 20, 30])

ax.set_title("Graph")
ax.set_xlabel("X")
ax.set_ylabel("Y")

plt.show()
```

Difference

Pyplot	Object-Oriented
Simple scripts	Complex visualization
Implicit current Axes	Explicit Axes
Easy for beginners	Better for reusable code
Less control over multiple Axes	Better control over layouts

Recommendation: Professional projects-এ Object-Oriented API বেশি ব্যবহার করো।

38. Matplotlib Best Practices

1. Always add chart titles.
2. Use meaningful axis labels.
3. Use legends for multiple datasets.
4. Avoid too many colors.
5. Choose appropriate chart types.
6. Use `tight_layout()` when necessary.
7. Use readable tick labels.
8. Save figures at suitable DPI.

9. Prefer Object-Oriented API.
10. Avoid misleading axis scales.
11. Use bar charts for category comparisons.
12. Use line charts for trends.
13. Use scatter plots for relationships.
14. Use histograms for distributions.
15. Use box plots for spread and potential outliers.

39. Real-World Project 1 — Sales Dashboard

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "month": ["Jan", "Feb", "Mar", "Apr", "May"],
    "sales": [100, 150, 120, 200, 250],
    "expenses": [60, 80, 70, 100, 120]
})

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# Sales line chart
axes[0, 0].plot(
    df["month"],
    df["sales"],
    marker="o"
)

axes[0, 0].set_title("Monthly Sales")

# Expense bar chart
axes[0, 1].bar(
    df["month"],
    df["expenses"]
)

axes[0, 1].set_title("Monthly Expenses")

# Sales histogram
axes[1, 0].hist(
    df["sales"],
    bins=5
)

axes[1, 0].set_title("Sales Distribution")

# Profit chart
profit = df["sales"] - df["expenses"]

axes[1, 1].bar(
    df["month"],
    profit
```

```
)

axes[1, 1].set_title("Monthly Profit")

plt.tight_layout()

plt.savefig("sales_dashboard.png", dpi=300)

plt.show()
```

40. Real-World Project 2 — Student Performance

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "student": ["Faruk", "Rahim", "Karim", "Sakib", "Hasan"],
    "marks": [80, 70, 90, 60, 85]
})

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Bar chart
axes[0].bar(
    df["student"],
    df["marks"]
)

axes[0].set_title("Student Marks")

# Histogram
axes[1].hist(
    df["marks"],
    bins=5
)

axes[1].set_title("Marks Distribution")

plt.tight_layout()

plt.show()
```

41. Real-World Project 3 — Stock Price Visualization

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "date": pd.date_range(
        "2026-01-01",
        periods=7,
        freq="D"
    ),
    "price": [100, 105, 103, 110, 115, 112, 120]
```

```
})

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(
    df["date"],
    df["price"],
    marker="o"
)

ax.set_title("Stock Price Trend")
ax.set_xlabel("Date")
ax.set_ylabel("Price")

fig.autofmt_xdate()

plt.show()
```

42. Interview Questions

Beginner

1. What is Matplotlib?

Python-এর data visualization library।

2. What is pyplot?

Matplotlib-এর plotting interface।

3. What is Figure?

পুরো chart canvas।

4. What is Axes?

যেখানে actual graph আঁকা হয়।

5. How do you display a graph?

```
plt.show()
```

6. How do you save a graph?

```
plt.savefig("chart.png")
```

7. How do you add title and labels?

```
plt.title("Title")
plt.xlabel("X")
plt.ylabel("Y")
```

Intermediate

8. Difference between bar and histogram?

Bar chart categorical data; histogram numerical distribution।

9. What is scatter plot?

দুটি numerical variables-এর relationship visualize করে।

10. What is subplot?

একটি Figure-এ multiple Axes তৈরি করা।

11. What is legend?

Graph-এর dataset বা symbols identify করে।

12. What is grid?

Graph-এর background reference lines।

13. What is DPI?

Dots Per Inch; image resolution-এর measure।

14. What is colormap?

Numerical values-কে colors-এ map করার system।

Advanced

15. Difference between Figure and Axes?

Figure হলো পুরো canvas; Axes হলো individual plotting area।

16. What is twin axis?

একই chart-এ আলাদা scale-এর দ্বিতীয় axis তৈরি করা।

17. What is animation?

সময় বা frame অনুযায়ী chart update করা।

18. What is a 3D plot?

Three-dimensional data visualize করা।

19. What is Object-Oriented Matplotlib?

Figure এবং Axes objects-এর মাধ্যমে graph তৈরি করার পদ্ধতি।

20. How do you optimize Matplotlib charts?

- Use suitable chart types.
- Avoid unnecessary elements.
- Use efficient plotting calls.
- Choose suitable DPI.
- Use appropriate figure dimensions.

43. Practice Problems

Beginner Level

1. Create your first line chart.
2. Create a bar chart of programming languages.
3. Create a horizontal bar chart.
4. Create a pie chart.
5. Create a histogram of student marks.
6. Add title, labels and legend.
7. Change line color and style.
8. Add grid to a graph.
9. Change figure size.
10. Save a graph as PNG.

Intermediate Level

1. Create multiple line plots.
2. Create grouped bar charts.
3. Create stacked bar charts.
4. Create scatter plots with color mapping.
5. Create box plots.
6. Create violin plots.
7. Create heatmaps.

8. Create 2×2 subplots.
9. Add annotations.
10. Customize ticks and axes.

Advanced Level

1. Create a 3D surface plot.
2. Visualize a normal distribution.
3. Create a rolling average chart.
4. Create a multiple Y-axis chart.
5. Build an animated sine wave.
6. Visualize correlation matrices.
7. Create a sales dashboard.
8. Visualize time-series data.
9. Create publication-quality charts.
10. Build an automated visualization report.

44. Complete Learning Roadmap

Level 1 — Beginner

- Introduction
- Installation
- Figure and Axes
- Line Plot
- Bar Chart
- Scatter Plot
- Histogram
- Pie Chart
- Title and Labels
- Legend and Grid

Level 2 — Intermediate

- Horizontal Bar
- Grouped Bar
- Stacked Bar
- Box Plot
- Violin Plot
- Area Chart
- Subplots
- Figure Size
- Axis Customization
- Annotations

Level 3 — Advanced

- Heatmap
- Colormaps
- Logarithmic Scale
- Multiple Y-Axes
- DateTime Plotting

- Statistical Visualization
- 3D Plotting
- Object-Oriented API
- Animation
- Interactive Visualization

Level 4 — Expert

- Complex layouts
- Custom plotting functions
- Advanced styling
- Performance optimization
- Large dataset visualization
- Scientific visualization
- Automated reporting
- Matplotlib integration with Pandas and NumPy

45. Recommended Study Routine

Time	Activity
30 minutes	Learn concepts
30 minutes	Write plotting code
30 minutes	Practice charts
30 minutes	Build projects

Recommended Projects

1. Student Result Dashboard
2. Monthly Sales Dashboard
3. Employee Salary Analysis
4. E-commerce Data Visualization
5. Stock Price Visualization
6. Weather Data Visualization
7. Machine Learning Training Curves

Final Advice

Matplotlib শেখার সময় বিশেষভাবে আয়ত্ত করো:

1. Figure and Axes
2. Line Plot

3. Bar Chart
4. Scatter Plot
5. Histogram
6. Subplots
7. Axis Customization
8. Annotations
9. Heatmap
10. Object-Oriented API

Final Goal: NumPy ও Pandas দিয়ে data prepare করে Matplotlib দিয়ে professional-quality visualization তৈরি করতে পারা।